

PROPOSAL PACKAGE | DOCUMENT 11

Presenter Briefing: Concepts and Glossary

Everything you need to understand to present this proposal with confidence, explained from first principles, with a glossary

Document	11 of 15
Audience	Presenter only. Not for distribution.
Version	1.0 (first-meeting edition)
Date	September 2026
Classification	Internal - presenter's copy

Contents

1. How to prepare in three hours.....	3
2. The big picture in plain words.....	3
3. The concepts, one by one.....	3
3.1 Large language model (LLM).....	3
3.2 Retrieval-Augmented Generation (RAG).....	3
3.3 Closed domain.....	4
3.4 Embeddings and vector search.....	4
3.5 Hybrid retrieval and re-ranking.....	4
3.6 Citations and the grounding check.....	4
3.7 Claims ledger (how contradictions are found).....	4
3.8 Decision checklists (how gaps are found).....	4
3.9 Audit log.....	4
3.10 Access control, SSO, RBAC.....	4
3.11 Model gateway.....	5
3.12 Data residency and sovereignty.....	5
3.13 Open-weight models and self-hosting.....	5
3.14 OCR.....	5
3.15 CRM.....	5
3.16 Pipeline and stages.....	5
3.17 Human-in-the-loop and the approval gate.....	5
3.18 Connectors, APIs, webhooks.....	5
3.19 Golden question set and evaluation.....	6
3.20 Tokens and model pricing.....	6
3.21 Infrastructure as code, containers, CI/CD.....	6
4. Numbers to remember.....	6
5. Phrases that land well.....	6
6. Things to avoid saying.....	7
7. Glossary (quick reference).....	7

1. How to prepare in three hours

1. Read this document once (60 minutes).
2. Read the Executive Summary (Document 01) and the deck (Document 02) with speaker notes (45 minutes).
3. Read the Q&A Playbook (Document 10) and say the short answers out loud (45 minutes).
4. Skim the Solution Architecture Part A (Document 03, sections 2-6) so you recognise every diagram (30 minutes).

You do not need to understand Part B of the architecture or the workbook formulas to present the first meeting. If the client's technical people ask detailed questions, the honest and professional answer is: "That is in the architecture document; our architect will walk your team through it in the detailed session."

2. The big picture in plain words

The client has a pile of important documents (contracts, approvals, financial models, letters) and a set of relationships (investors, partners, customers). Two problems:

- **Nobody can hold all the documents in their head.** Figures get quoted from an old draft. A decision is taken before the supporting paper arrives. Two documents disagree and nobody notices until it matters.
- **Relationships slip.** Follow-ups get forgotten. Nobody knows who last spoke to an investor. And the client is rightly nervous about software emailing important people on its own.

We are proposing two tools on one foundation:

- A **reading assistant for management** that answers questions only from their documents, shows where the answer came from, flags when documents disagree, and lists what is still missing before a decision.
- A **relationship tracker** that keeps every contact, stage and follow-up in one place, and that will not send any message unless a person approves it.

The foundation (login, shared contact list, document store, AI gateway, audit log) is built once, so both tools, and future tools, sit on it.

3. The concepts, one by one

3.1 Large language model (LLM)

A program trained on enormous amounts of text that can read and write language. Examples: Claude (Anthropic), GPT (OpenAI), Gemini (Google), and open-weight models like Qwen and GLM that anyone can host. An LLM is very good at reading a passage and answering questions about it, and at turning messy text into structured data. It is **not** a database and it does not "know" the client's business. Left alone, it will guess. Everything in our design exists to stop it guessing.

3.2 Retrieval-Augmented Generation (RAG)

The standard way to make an LLM answer from private documents. Two steps: **retrieve** the few passages relevant to the question, then **generate** an answer using only those passages. Think of an analyst who is handed the three relevant pages and told "answer from these, and cite the page". The quality of a RAG system is mostly the quality of its retrieval and its discipline about citations.

3.3 Closed domain

The system is limited to the client's loaded documents. No internet, no general knowledge. If the answer is not in the documents, the correct behaviour is to say so.

3.4 Embeddings and vector search

To find passages by meaning rather than by exact words, each passage is converted into a list of numbers (an "embedding") that captures its meaning. Similar meanings produce similar numbers. A "vector database" stores these numbers and finds the nearest ones to the question. This is how an English question finds an Arabic clause. We use PostgreSQL with the pgvector extension for this; no separate product needed at the start.

3.5 Hybrid retrieval and re-ranking

Meaning-based search is weak on exact numbers, names and codes; keyword search is weak on paraphrase. We use both and merge the results ("hybrid"). Then a "re-ranker" model reads the question together with each candidate passage and orders them properly. The model then sees only the best 8-12 passages.

3.6 Citations and the grounding check

Every sentence of an answer must point to a document and page. After the answer is drafted, a second, separate pass checks each sentence against its cited passage and removes anything unsupported. This is the single most important trust feature. In the meeting, say: "It shows its work, and a second check marks its homework."

3.7 Claims ledger (how contradictions are found)

When a document is loaded, the system extracts every fact into a table row: entity (who or what), attribute (contract value, start date, term), value, unit, page, and the exact quote. Comparing rows about the same entity and attribute across documents finds disagreements. Search alone cannot do this because it looks at a few passages at a time; the ledger looks at everything.

3.8 Decision checklists (how gaps are found)

Management tells us once what evidence each decision needs. The system checks what is present, missing or contradicted for each decision. A gap is "we need a signed invoice to release this payment and there is none in the documents".

3.9 Audit log

A record of who did what and when that cannot be silently altered (each entry is chained to the previous one with a cryptographic hash). Regulators and auditors like this; so do executives.

3.10 Access control, SSO, RBAC

SSO (single sign-on): staff log in with their existing company account (usually Microsoft). **RBAC** (role-based access control): what a person can see depends on their role. Per-document access lists mean the assistant only searches documents the user is allowed to open.

3.11 Model gateway

One internal door through which all AI requests pass. It decides which model to use (based on cost, document sensitivity, or country rules), logs usage and cost, and lets us swap providers without changing the application. This is our answer to "are you locking us into one AI vendor?"

3.12 Data residency and sovereignty

Residency: where data is stored. **Sovereignty:** which country's laws govern it and whether it ever leaves. GCC clients care a lot. Our position: data at rest stays in a GCC region from day one; AI processing can be routed by policy, including to a model hosted inside the country (at higher cost). See Document 08 for the laws.

3.13 Open-weight models and self-hosting

Some strong models (Qwen, GLM, Gemma, DeepSeek) are published for anyone to run on their own servers. Running one "in country" on rented GPUs is how we achieve full sovereignty. They are slightly behind the best commercial models and cost more to operate, which is why we recommend deciding after the pilot.

3.14 OCR

Optical character recognition: turning a scanned image into text. Needed for scanned contracts and Arabic paperwork. We use a cloud OCR service available in the UAE region, or self-hosted OCR for the sovereign option.

3.15 CRM

Customer relationship management software: a structured record of organisations, people, the state of each relationship ("stage"), interactions, and tasks. Ours is purpose-built and small, with three category pipelines and a GCC-specific one.

3.16 Pipeline and stages

A pipeline is the sequence of stages a relationship moves through. Our GCC pipeline example: introduction through an intermediary; first meeting; NDA; qualification; active discussion; term sheet or MoU; regulatory or board approval; signed; dormant. Stages are configurable by category.

3.17 Human-in-the-loop and the approval gate

The rule that no outbound message is sent without a named person approving it, and that the drafter cannot approve their own message ("four-eyes"). We implement it in the database, not just in the interface, so it cannot be bypassed through the system.

3.18 Connectors, APIs, webhooks

Connector: a built-in link to another system (SharePoint, Google Drive, a mailbox). **API:** a programmable door other software can use to read or write records. **Webhook:** a notification the platform sends when something happens ("a finding was created"). Together they are how real data plugs in later and how other systems integrate.

3.19 Golden question set and evaluation

A list of real questions with known correct answers and pages, agreed with the client. We score the system on it after every change. It turns "is it accurate?" into a number and protects against regressions. This is how we make promises that are measurable.

3.20 Tokens and model pricing

Models are priced per million "tokens" (roughly three-quarters of a word). A 30-page contract is about 15,000 tokens. Prices in 2026: Claude Opus 5 at USD 5 per million input tokens and USD 25 per million output; Claude Sonnet 5 at USD 2 and USD 10. This is why pilot AI cost is only hundreds of dollars a month.

3.21 Infrastructure as code, containers, CI/CD

Infrastructure as code (Terraform): the cloud set-up is written as files, so it is repeatable and auditable. **Containers** (Docker): the software is packaged so it runs identically anywhere. **CI/CD**: automated tests and deployments on every change. Together they are why the same code runs at every tier and in any cloud.

4. Numbers to remember

Fact	Value
Integrated working demo	Week 8-9
Production go-live	Week 20
Pilot run cost	Roughly USD 150-400 per month plus build effort
Departmental tier run cost	Roughly USD 1,500-5,000 per month
Enterprise / sovereign tier run cost	Roughly USD 15,000-60,000 per month
Accuracy targets at pilot exit	95% answer faithfulness; 95% numeric extraction precision; 90% seeded contradictions detected
Core team	3-4 people plus part-time specialists
Client owns	Code, cloud accounts, data, from day one

5. Phrases that land well

- "It answers only from your documents, shows the page, and says 'not in the documents' when that is the truth."
- "Contradictions need structure, not just search."
- "Nothing leaves the building without a human's name on it."
- "Residency is a policy setting, not a rebuild."
- "We measure accuracy on your questions, not ours."
- "You own everything from day one."

6. Things to avoid saying

- Any absolute accuracy claim ("never wrong", "100%").
- "Certified compliant" (say "designed to support compliance; your legal team confirms the lawful basis").
- A single all-in price before discovery (present the tiers and the model).
- Vendor bashing.
- Technical detail you are not sure of; use the fall-backs in Document 10.

7. Glossary (quick reference)

Term	Meaning
ACL	Access control list: who may open a document or record
API	Application programming interface: a way for software to talk to software
Chunk	A passage of a document (a few hundred words) stored for retrieval
Claims ledger	Table of extracted facts used to detect contradictions
Cross-region inference	Cloud AI processing that may route a request to another region for capacity
DIFC / ADGM	Dubai and Abu Dhabi financial free zones with their own data protection laws
Embedding	Numeric representation of meaning used for semantic search
Entra ID	Microsoft's identity service (formerly Azure Active Directory)
Four-eyes	A second person must approve; drafter cannot self-approve
GPU	Graphics processing unit: the hardware that runs AI models
Grounding	Verifying each answer sentence against its cited source
HNSW	An indexing method for fast vector search
IdP	Identity provider (Microsoft Entra ID, Okta, Google)
LLM	Large language model
MFA	Multi-factor authentication
OCR	Optical character recognition (scans to text)
OIDC	OpenID Connect: the standard for single sign-on
PDPL	Personal Data Protection Law (Saudi Arabia; UAE also uses the term)
pgvector	PostgreSQL extension for vector search
RAG	Retrieval-augmented generation
Re-ranker	Model that orders retrieved passages by relevance
RLS	Row-level security: database-enforced access filtering

Term	Meaning
RTO / RPO	Recovery time objective / recovery point objective for disasters
SDAIA	Saudi Data and AI Authority (PDPL regulator)
SIEM	Security information and event management system
SLA	Service level agreement
SSO	Single sign-on
Token	Unit of text used for model pricing (about 0.75 words)
vLLM	Open-source server for running open-weight models on GPUs
Webhook	Notification sent by the platform when an event happens